A model for static conflict analysis of management policies

A Majidian

Management of today's distributed systems is becoming increasingly complex. There is an obvious requirement for a flexible mechanism to help manage such systems. Rule-based management is one such mechanism. However, in order for rule-based management to become widely usable a method is required by which conflicts between management policies (defined as rules) can be identified and resolved. This paper creates a set theoretic model for rules as a tri-tuple of the relationship between the subject, action and target of a policy. It also identifies two classes of policy set — 'syntactically easy policy set' (SEPS) and 'syntactically non-easy policy set' (SNEPS). SEPSs are policies which are sets of all the Cartesian products of its subjects, actions and targets, whereas SNEPSs are only a subset of that Cartesian product. Conflict analysis of SEPSs has been handled in other papers; this paper addresses conflict analysis of SNEPSs. A method for resolving conflict is suggested. The paper also raises some issues that arise when considering a database of policies.

1. Introduction

As computing becomes a more and more distributed/network-centric activity, the management of these systems becomes increasingly complicated. In distributed systems, shared data/objects may be the only things that bind together many different sub-systems (see Shuey et al [1]). A distributed application may be spread across several computer systems dispersed globally. The notion of a unified authority to manage the overall activities of the system does not exist, since, in practice, a distributed system is a multitude of software programs operating on a variety of platforms. Rule-based management is a radical approach developed by, among others, Professor Morris Sloman and his team at Imperial College, to address the challenges of modern distributed systems management. For rule-based management to become a norm in the management of distributed systems there is a series of issues that need to be addressed. Prominent among these issues is the need for a distributed management scheme, which itself is distributed, to have the ability to detect conflict between rules statically at compile time.

2. Rule-based management

To understand rule-based management one has to understand the concept of object space. The notion of object space, though not mentioned in an explicit fashion in any literature relating to rule-based management, is nonetheless implied and is central to an understanding of the concept of rule-based management.

- Definition

  The object space is the management view of the set of all objects on a distributed platform upon which a structure has been imposed by the management.

An object space can have a domain structure imposed upon it. The domain structure partitions the object space into non-mutually-exclusive spheres of management control and creates a hierarchical structure, very much like the Unix directory system. A domain references objects and domains of objects (i.e. sub-domains) and can have one or more parent domains. It is possible and permitted for cycles to occur. Sloman et al [2] define the domain as follows.

- Definition

  A domain is an object that represents a collection of objects which have been explicitly grouped together to apply a common management policy.

Policies are used to dictate a particular set of behaviour on a distributed system, without hard coding this behaviour into the structure of the system itself. The syntax of the policies which are considered in this paper is that described in Marriot [3]. The policies are divided into two general categories of authorisations and obligations and these two

categories are in turn sub-divided into positive and negative forms. An authorisation policy specifies what a manager object can do (in positive mode) and what a manager object is forbidden to do (in negative mode). An obligation policy cites what a manager object must or must not do. For a more detailed explanation of both syntax and semantics of the rule-based policy management examined in this paper, see Marriot [3], Sloman [4], and Lupu and Sloman [5].

To begin, consider the simplified syntax of a policy, either authorisation or obligation (for the complete syntax, see Marriot [3]).

- Policy:

  identifier mode [trigger] subject {action} target [constraint];

  Whereas the curly brackets and semicolon are lexical tokens, the square brackets contain optional items:

  identifier — the name assigned to the policy

  mode — either $A+$ or $A-$ or $O+$ or $O-$ [1]

  [trigger] — descriptions of events which trigger an obligation policy

  subject, target — objects

  [constraint] — a predicate

If the syntax of the policy is collapsed and only its subject, action and target are considered (see Lupu and Sloman [5], and Moffett and Sloman [6] where similar strategies have been adopted), policies can be considered as a sub-set of the Cartesian product of the subject, action and target sets. Basically what we are suggesting is:

> let σ be the set of all subjects, α the set of all actions and τ the set of all targets; also let p be any arbitrary policy, then:
>
> $p \in \sigma \times \alpha \times \tau$

Policies can be grouped together to form a set of policies, as management policies often are. Hence, we can talk of the set of policies P where $P \subseteq \sigma \times \alpha \times \tau$ for any element $p \in P$ where $p = (x, y, z)$, which means that object x can do action y on object z.

The syntax above provides a facility to group the policies together into a set of policies (this is only syntactic sugar — however, since Lupu and Sloman [5, 7] exploit this facility, it is necessary to put some formal touches to it as the notation will be referred to later). Using the syntax, policies are often presented as $P = \langle S, A, T \rangle$ which says that P consists of $|S| \times |A| \times |T|$ [2] number of policies, e.g. (in positive authorisation mode) $\langle S, A, T \rangle$ means each element of S can do each action in A on each element of T. Formally it can be stated as:

$$\langle S, A, T \rangle = \{(s, a, t) | s \in S, a \in A, t \in T\}$$

$$\text{i.e.} \langle S, A, T \rangle = S \times A \times T$$

It should be noted that:

$$\langle S, A, T \rangle \subseteq \sigma \times \alpha \times \tau$$

3. Occurrence of conflict between policy sets

It is instructive to recognise that policies need not only be grouped together as the entire Cartesian product of subject, action and target sets, since a subset of the Cartesian product is an equally valid set of policies. For example $P = \{(a, b, c), (d, e, f)\}$ is a valid set of policies, although it is not the Cartesian product of any subject, action and target sets. However, the syntax of the language in Marriot [3] does not provide a neat and easy way of grouping together at input time sets of policies that are not the entire Cartesian product. It also does not provide a syntactic facility to assign a common identifier to two or more policies defined separately — this could perhaps be considered as an expansion of the syntax of the language. These kinds of policy sets are called 'syntactically non-easy policy sets' (SNEPSs) and the grouping of the policies offered by the current syntax of the language as 'syntactically easy policy sets' (SEPSs). To state a mathematical relationship between SNEPS and SEPS, consider a set S of subjects, a set A of actions and a set T of targets. The set of all of the policy sets that can have as their subject an element from S, as their action an element from A, and as their target an element from T, is $\wp(S \times A \times T)$, i.e. the power set of the Cartesian product $S \times A \times T$.

Therefore the total number of policy sets that can be defined on this product, including the empty policy set $\varnothing$ is:

$$|\wp(S \times A \times T)| = 2^{|S \times A \times T|} = 2^{(|S| \times |A| \times |T|)}$$

SNEPS represents $2^{|S \times A \times T|} - 1$ policy sets that can be defined on $S \times A \times T$ and SEPS represents the remaining one, namely that of $S \times A \times T$ itself, that is:

$$\text{SEPS} \cup \text{SNEPS} = \wp(S \times A \times T)$$

Later on in this paper, it will be shown that these SNEPS do not fit nicely into the analysis provided for SEPS by Lupu and Sloman [5, 7].

[1] Where A is authorisation, O is obligation, + is positive and − is negative.

[2] Where $|X|$ is the size of set X

Since the purpose of this set theoretic modelling of policies is to capture the notion of conflict, it is first necessary to explore exactly what conflict is and thereby offer a definition. To this end consider the following three pairs of positive-negative policies:

- manager M can read file F
  manager M cannot read file H

- manager M can view file F
  manager M cannot ftp file F

- manager M can read file F
  manager N cannot read file F

Obviously, none of the pairs of policies above is contradictory (it is, of course, always possible to put a slant to the meaning of a pair of policies expressed in natural language and find some conflict). Conflict in this model can only occur when the subject and the target remain the same but the action is contradictory (there are other forms of conflict mentioned in Lupu and Sloman [5, 7]) but with this model the interest only lies in the modal conflict — see the subsequent discussion on the subject, for example:

- manager M can read file F
  manager M cannot read file F

To be more precise the conflict occurs when two policies with the same subject and target are:

1. authorised ($A+$) and forbidden ($A-$) to do the same action,

2. obliged ($O+$) and required not ($O-$) to perform the same action,

3. obliged ($O+$) but not authorised ($A-$) to do the same action.

The fourth option of being authorised ($A+$) but not obliged ($O-$) to do the same action obviously is not a conflict.

Lupu and Sloman [5] state: '... the modality conflicts are inconsistencies in the policy specification which may arise when two or more policies with modalities of opposite sign refer to the same subject, action and target,' and go on: '... this occurs when there is a triple overlap between sets of subjects, actions and targets as shown in Fig 1 and so can be determined by syntactic analysis of policies ...' (this refers to the use of triple overlap SEPS) — of course this would have been an error with SNEPS since there could be a triple

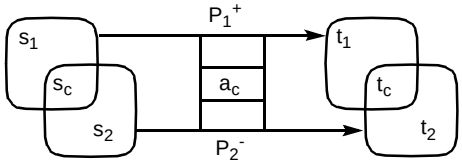

Fig 1 Overlapping subjects, targets and actions (diagram from Lupu and Sloman [5]).

overlap without any conflict. In general, because policy sets do not apply to every element of the Cartesian product of the subject, action and target sets on which they are operating, it would be an error to disentangle the relationships to their constituent subject, action and target parts and to analyse them as though they were separate sets.

To illustrate the point, consider the following two sets of policies — P and N, positive mode and negative mode respectively. (As was defined more succinctly above, only three combinations of positive versus negative policies create conflicts; from now on, therefore, whenever mention is made of 'conflict', it should be interpreted as referring to one of the three categories itemised above — see also below for formal definition of conflict.)

$$P = \{(a, b, c), (e, b, c), (e, d, f)\}$$

$$N = \{(a, g, c), (h, d, i), (h, g, c)\}$$

Obviously there is no conflict since $P \cap N = \varnothing$, but the SEPS model would not work here since separation of the subjects, actions and targets would mean:

$$P = \{a, e\} \times \{b, d\} \times \{c, f\} \quad \text{and}$$

$$N = \{a, h\} \times \{g, d\} \times \{c, i\}$$

while P and N are in fact only subsets of these products. Figure 2 shows the overlap of elements between P and N. Essentially it is the same shape as Fig 1 above.

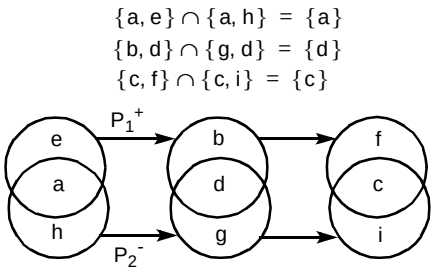

Fig 2 $P = \{(a, b, c), (e, b, c), (e, d, f)\}$
$N = \{(a, g, c), (h, d, i), (h, g, c)\}$
Two policies can intersect in respect of their constituent parts, but have no conflict.

The problem arises because set theoretically the definition of (a, b) is $\{\{a\}, \{a, b\}\}$; therefore $\{(a, b)\} = \{\{\{a\}, \{a, b\}\}\}$ and since $(a, b, c) = (a, (b, c))$ then $\{(a, b, c)\}$ is the rather convoluted set $\{\{\{ a\}, \{a, \{\{b\}, \{b, c\}\}\}\}\}$ (see Goldrei [8]). For a simple and elegant proof, see Halmos [9]. In general, for converting tuples to sets, the method shown above has to be used. If, and only if, all elements of the Cartesian product are present in the set of tuples can the (policy) set be disentangled to the constituent parts and presented as the Cartesian product of the constituent parts.

Now, to formally define the conflict, all the policies within the system are grouped into sets of policies according to their mode.

Let P_1 be the set of all positive authorisations ($A+$), P_2 be the set of all negative authorisations ($A-$), P_3 be the set of all positive obligations ($O+$) and P_4 be the set of all negative obligations ($O-$).

- Definition

  Conflict arises when $\exists p = (s, a, t)$ such that any one of the following conditions is satisfied:

  1. $p \in (P_1 \wedge P_2)$ or
  2. $p \in (P_3 \wedge P_4)$ or
  3. $p \in (P_2 \wedge P_3)$

There are times that conflict can be resolved in a policy specification by means of defining precedence. One way of defining precedence is that a set of policies which are more specific overrides a set of policies which are less specific. The next section will show how this can be achieved, but, first, an explanation of how the separation of a policy tuple into its constituent sets as mentioned above does not work with SNEPS and can lead to erroneous detection of conflict which in turn can make the whole exercise superfluous.

3.1 Conflict resolution through precedence assignment

Lupu and Sloman [5] try to resolve the conflicts for SEPS using precedence. They propose conflict resolution using a technique called domain nesting by stating: 'The analysis for conflicts of a set of policies enumerates all the subject, action and target tuples which have different sets of policies applying to them. Consider the policies P_1 and P_2 represented in Fig 1 [this diagram is from Lupu and Sloman], P_1 being positive and P_2 being negative. The overlapping areas are called s_c, a_c and t_c for common subjects, actions and targets.' The paper then goes on to identify the three discrete regions below:

- $\langle s_1 - s_c, a_1 - a_c, t_1 - t_c \rangle$
- $\langle s_c, a_c, t_c \rangle$, where the actual 'conflict' lies
- $\langle s_2 - s_c, a_2 - a_c, t_2 - t_c \rangle$

Having identified these regions, the crux of the approach is that in some situations it is possible to give precedence to a set of policies which apply to a more specific set of subjects, targets or both. It utilises a pictorial approach using Venn diagrams (see Fig 3) to identify where conflicts can and cannot be resolved through giving precedence to inner domains (see also Lupu and Sloman [7] in which the argument is repeated).

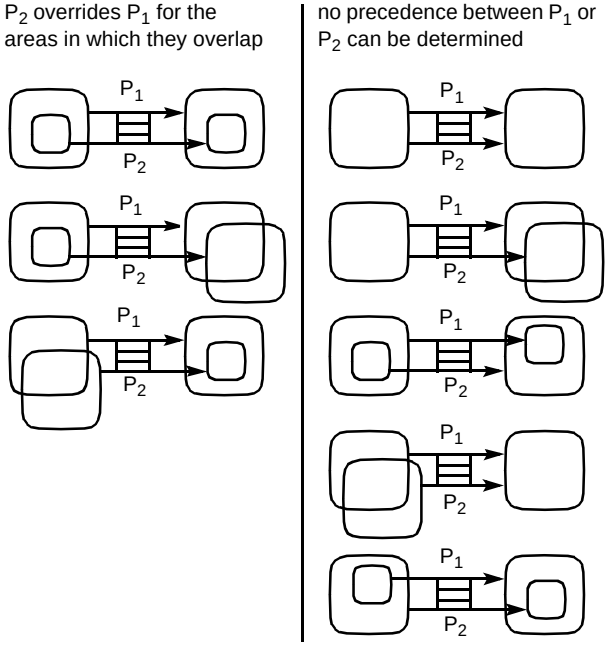

Fig 3 The precedence between policies (diagram from Lupu and Sloman [5]).

However, this approach would not work if it were applied to SNEPS. The problem this approach faces is that, by separating out the constituent parts of a policy set, a lot of non-conflict situations are detected that do not require a flagging of any conflict or resorting to precedence rules.

To illustrate that this approach can lead to detecting a non-conflict as a conflict for SNEPS, two Venn diagrams have been selected. One Venn diagram is from those on the left of Fig 3 with conflict which it is stated can be resolved through precedence consideration. The other Venn diagram is from those on the right, for which the system cannot resolve the conflict. Concrete subject, action and target elements have been added.

The following policy is defined on Fig 4(a):

let $p_1^+ = \{(a, r, b), (c, r, d)\}$ and $p_2^- = \{(c, r, b)\}$,

here $S_1 = \{a, c\}$ $T_1 = \{b, d\}$

$S_2 = \{c\}$ $T_2 = \{b\}$

$S_2 \subset S_1$ $T_2 \subset T_1$

However, there are no conflicts to be resolved, and if the following policy is defined on Fig 4(b):

let $p_1^+ = \{(a, r, b), (c, r, d)\}$ and

$p_2^- = \{(a, r, d), (c, r, e)\}$

here $S_1 = \{a, c\}$ $T_1 = \{b, d\}$

$S_2 = \{a, c\}$ $T_2 = \{d, e\}$

$S_1 = S_2$ $T_1 \cap T_2 = \{d\} \neq \varnothing$

and, again, there are no conflicts to be flagged up.

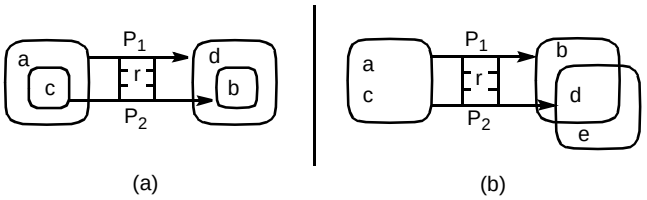

Fig 4 In (*a*) $p_1^+ = \{(a, r, b), (c, r, d)\}$ and $p_2^- = \{(c, r, b)\}$, in (*b*) $p_1^+ = \{(a, r, b), (c, r, d)\}$ and $p_2^- = \{(a, r, d), (c, r, e)\}$

Here, it is worth mentioning that the papers [2, 3] do remark that it is only potential conflict that is being dealt with, but, when reference is made to potential conflict in both papers, it means that after detection (through splitting the policy into its constituent parts) it can be resolved by using precedence rules. It is definitely not about flagging up conflict for everything that has an overlap in subjects, actions or targets.

3.2 A simple solution for the model

As has been illustrated, any breaking up of the tri-tuple for SNEPS, as shown above, to its constituent parts and analysing the policy set in that manner can result in detecting non-conflicting situations as potential/real conflicts needing resolution or reporting back to the system administrator. The easiest approach would be to consider a set of policies as a set of tri-tuples so each element of the set has the form (*s*, *a*, *t*), rather than three separate sets of subject, action and target. With this approach two sets of policies with opposite modalities (as described in the definition of the conflict) can pictorially have only one of three relationships, as shown in Fig 5.

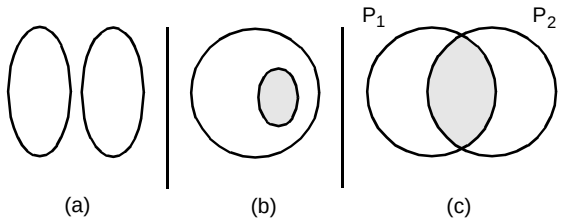

Fig 5 Policy relationships.

The only three possibilities of conflict are — (a) there is no conflict, (b) the inner set has precedence, (c) $P_1 \cap P_2$ region is a conflict area which cannot be resolved generically, and only in some particular/subject-specific situation can be resolved by some meta-policy.

3.3 Bi-policy set approach and its limitations

Conflict resolution, as it has been discussed above, compares sets of policies pair-wise. Even in the restricted model of SEPS, to look at sets of policies pair-wise produces a different picture from when the sets of policy sets, i.e. a database of policies, are considered. Examine the following example:

$$p_1^+ = \langle \{a_1\}, \{b\}, \{c\} \rangle$$

$$p_2^- = \langle \{a_1, a_2, a_3\}, \{b\}, \{c\} \rangle$$

which pictorially can be represented as in Fig 6 and where presumably P_1^+, being more specific, can override P_2^-.

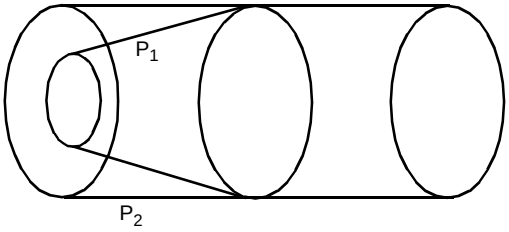

Fig 6 Limited view, implying P_1 should take precedence over P_2.

Now assume there is another policy P_3^+ defined as:

$$P_3^+ = \langle \{a_1, d_1, d_2\}, \{b\}, \{c\} \rangle$$

The combined pool of policies in P_1^+ and P_3^+ can change the picture to that of Fig 7. Moreover, it is not now possible to determine which of P_1^+ and P_3^+ or alternatively P_2^- has precedence due to being more specific.

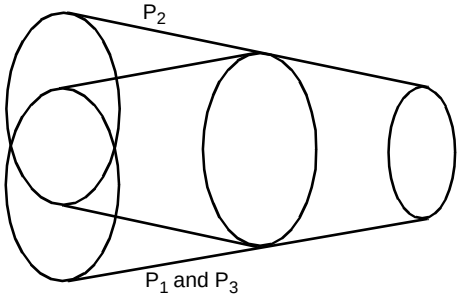

Fig 7 Full view, where it is not possible to determine precedence.

3.4 Some examples of policies which need more complicated models

Below are described a couple of scenarios that will demonstrate that on occasions it is necessary to resort to a

more complicated model for capturing the semantics of the policies and thereby detecting conflict.

- Consider the following two policies seemingly contradictory to the tri-tuple model which does not consider temporal aspects (although the syntax clearly does have such a provision through the use of constraints — see syntax in Marriot [3]):

  1. members of domain *A* can read files in domain *B* between 9 am and 5 pm,

  2. members of domain *A* cannot read any file in domain *B* between 5 pm and 9 am the following day.

  A solution to this problem would be to make discrete time as yet another set and to expand the original model from a tri-tuple to a quad-tuple model, i.e. (*s*, *a*, t_r, t_i)

- For the second scenario, consider a situation within an organisation. Due to some resource constraint the following policy is implemented:

  1. members of group *A* are not allowed to use a certain facility (e.g. printing), where $A \subseteq B$.

  Later, as the problem is sorted out, the following policy is fed into the system:

  2. all members of *B* can use the printer.

  Clearly, the intended policy after instalment of the second policy is to override the first policy. While, if we insist (through some meta-policy decision) that the more specific policy should override the less specific policy, we will end up with the first policy.

4. Some further issues of interest

The main characteristic of these approaches (described here and in Lupu and Sloman [5]) is that they view the object space as a flat set — they do not utilise any inherent hierarchical structure that exists within the domain. Basically the conflict analysis techniques presented both by this paper and by Lupu and Sloman [5] have a flat view of the object space (the operators that are used are set inclusion, subset, intersection, etc, which set theoretically are defined on flat sets [3]). These approaches, though they have the considerable advantages of being simple, elegant and efficient in terms of time and space, are discarding information which is contained in the hierarchical structure of the domain on which they are operating. These approaches impose a reliance on the relative size of the policy set for the assignment of precedence. However, assigning precedence to the smaller set of policies in time of conflict (as adopted both in this paper and by Lupu and Sloman [5] though they refer to it as giving precedence to the more specific policy in its tri-tuple approach) imposes an unrealistic constraint on the system designers and system managers, namely that, if they want their policies to be effective, they have to add their policies to the rule-base engine in ever smaller sets, so that the new smaller sets would take precedence over the existing larger sets.

[3] It is important to understand that, set theoretically, sets are flat objects. Sets in the context of a programming language can of course be implemented as arrays, link lists, trees or even more sophisticated structures; however, the implementation structure does not make them non-flat sets. The implementation is irrelevant since only set theoretic operators like union, insertion, etc, are used which assume no hierarchy.

5. Conclusions and future work

As can clearly be seen, the static analysis of conflict in rule-based management is fraught with difficulties and any decision to resolve conflict in any particular manner can throw up its own anomalies. What needs to be emphasised at this point is that assigning precedence to a set of policies is a meta-policy decision; but how well this assignment can be reflected in a model is dependent on the quality of the model and the functions which map the policies into the model.

A further direction which needs to be exploited is the assignment of precedence to objects with respect to their position in the domain hierarchy. However, domains being defined with cycles can pose its own problem (e.g. if higher precedence is given to sub-domain members and the sub-domain happens to be, through a cycle in the domain structure, also a parent domain of the current domain, then this class of precedence assignment cannot be valid). A solution might be to impose some restriction on the structure of the domain. Obviously these restrictions would have their own consequences — research has to be done on the trade-offs between such restrictions and the suitability of the restricted domain structure for expressing a rule-based management system.

Acknowledgements

We acknowledge the assistance and contributions of Professor Morris Sloman and Emil Lupu to the ideas presented in this paper. We would also like to thank our BT colleagues Paul McKee and Arabella Maude for their reviews of this paper and invaluable comments. Finally we would also like to thank Ian Henning for his assistance.

References

1 Shuey R L, Spooner D L and Frieder O: 'The architecture of distributed computer systems', Addison-Wesley (1997).

2 Sloman M and Twidle K: 'Domains: a framework for structuring management policy', in Sloman M (Ed): 'Network and distributed systems management', Addison-Wesley (1996).

3 Marriot D: 'Policy service for distributed system', PhD thesis (1997).

4 Sloman M: 'Policy driven management for distributed systems', Journal of Network and Systems, Plenum Press, 2, No 4 (1994).

5 Lupu E and Sloman M: 'Conflict analysis for management policies', Proceeding of Vth International Symposium on Integrated Network Management IM'97 (formally known as ISINM), San-Diego, Chapman & Hall (May 1997).

6 Moffett J D and Sloman M: 'Policy conflict analysis in distributed system management', Journal of Organisational Computing (April 1993).

7 Lupu E and Sloman M: 'Towards a role-based framework for distributed systems management', Journal of Network and Systems Management, 5, No 1, Plenum Press (1997).

8 Goldrei D: 'Classic set theory', Chapman & Hall (1996).

9 Halmos P R: 'Naïve set theory', Van Nostrand (1960).

Andrei Majidian received his BSc Honours degree in Mathematics and Computer Science from North London Polytechnic in 1990, and his MSc in the Foundations of Advanced Information Technology from Imperial College in 1992. He worked as a lecturer in AI at the University of Westminster before joining BT in 1997.

After previously working in the area of distributed systems management, he has recently joined the systems research unit.